Tam olarak öyle denemez, çünkü bu durum kodun çalıştığı ortama (OS olup olmamasına) göre değişir.

Eğer projeniz **Linux, Windows veya macOS** gibi modern bir işletim sistemi üzerinde çalışıyorsa, malloc kullanmasanız bile yazdığınız manuel bellek yönetimi kodu arka planda **hala işletim sisteminin bellek özelliklerini ve sanal bellek (virtual memory) mimarisini kullanır.**

İşletim sistemli ve işletim sistemsiz ortamlarda bu durumun nasıl işlediğini şu şekilde ayırabiliriz:

1. İşletim Sistemi Olan Bir Ortamda (Örn: Linux) malloc Kullanmamak

Siz malloc fonksiyonunu çağırdığınızda, C standart kütüphanesi (glibc) sizin adınıza işletim sisteminden büyük bellek blokları ister. Eğer malloc kullanmayıp belleği kendiniz yönetmek isterseniz, işletim sistemini tamamen devre dışı bırakamazsınız. Sadece aradaki "C kütüphanesi" katmanını aradan çıkarıp **doğrudan işletim sisteminin kapısını çalmış olursunuz**:

- **Sistem Çağrıları (System Calls):** Kendi bellek yönetim sisteminiz için işletim sisteminden ham (raw) bellek talep etmek zorundasınızdır. Bunun için Linux'ta brk, sbrk veya daha yaygın olarak mmap sistem çağrılarını kullanırsınız.
- **Sanal Bellek Yönetimi:** İşletim sisteminin size verdiği adresler gerçek fiziksel RAM adresleri değildir, **Sanal Adreslerdir (Virtual Addresses)**. Sayfalama (Paging), bellek koruması (Memory Protection - örn: segmentation fault) ve RAM yetmediğinde diski kullanma (Swap) gibi tüm OS özellikleri arka planda çalışmaya devam eder.

Yani bu senaryoda işletim sisteminin özelliklerini kullanmamış olmazsınız; sadece **işletim sisteminin size ham olarak verdiği belleği kendi içinizde nasıl dağıtacağınızı (manuel olarak) yönetmiş olursunuz.**

2. İşletim Sistemi Olmayan Bir Ortamda (Bare-Metal / Gömülü Sistem)

İşte bu senaryoda sorunuzun cevabı **evet** olur. Bir mikrodenetleyicide (örneğin bir ARM Cortex-M veya Arduino üzerinde) çalışıyorsanız:

- Ortamda bir işletim sistemi olmadığı için sanal bellek, sayfalama veya hafıza koruması yoktur.
- Yazdığınız manuel bellek yönetimi kodu doğrudan **fiziksel RAM adresleri** (örneğin 0x20000000) üzerinde çalışır.
- Belleğin sınırını aşarsanız sizi durduracak bir işletim sistemi ("Segmentation Fault" verecek bir mekanizma) yoktur; kodunuz doğrudan yanındaki başka bir veriyi bozar veya işlemci kilitlenir.

Özet Karşılaştırma

Durum	malloc Kullanımı	Manuel Bellek Yönetimi (Kendi malloc'unuz)
İşletim Sistemli Ortam (Linux/Windows)	Standart kütüphane OS'ten (mmap/brk ile) bellek alır, size küçük parçalar halinde dağıtır.	OS'ten ham belleği (mmap ile) siz istersiniz, kendi uygulamanız içinde parçalara bölersiniz. **(OS özellikleri hala devrededir)**
İşletim Sistemsiz Ortam (Bare-Metal)	Genellikle hiç yoktur veya derleyicinin çok basit bir gömülü kütüphane fonksiyonudur.	Doğrudan fiziksel RAM adres haritasını (Linker Script) kullanarak belleği tamamen kendiniz yönetirsiniz. **(OS özellikleri yoktur)**

Geliştirmek istediğiniz bu manuel bellek yönetimi **hangi ortamda** çalışacak?

- Linux/Windows gibi bir **işletim sistemi üzerinde** performans/fragmantasyon için mi?
- Yoksa bir mikrodenetleyici üzerinde **yalın donanım (bare-metal)** projesi için mi?

Amacınızı paylaşırsanız, o ortama özel (mmap kullanımı veya sabit blok havuzları gibi) bir **kod mimarisi örneği** üzerinden ilerleyebiliriz.